Code Verification in Rust with Verus

Programs as mathematical objects; correctness as a theorem

When an algorithm becomes a program

A proof about an algorithm is a starting point.
What theorem does its implementation satisfy?

- Specify the domain, hypotheses, and desired conclusion.
- Model execution as a relation between states.
- Prove that every permitted execution preserves the claim.

Today's example

A bounded successor: increment an integer until it reaches a limit.

The route

Introduction

What is code verification

Z3

Verus

Verification as LLM Guardrails

A bounded successor: the intended mathematics

For a fixed $L \in \mathbb{N}$, define

$$f_L : \{0, \ldots, L\} \longrightarrow \{0, \ldots, L\}, \qquad f_L(x) = \min(x + 1, L).$$

A tempting implementation

$$g_L(x) = \begin{cases} x + 1 & x \leq L, \\ x & x > L. \end{cases}$$

At $x = L$, we obtain $g_L(L) = L + 1$: the claimed codomain is violated.

We want a proof of $\forall x \in \{0, \ldots, L\},\ g_L(x) = f_L(x)$—or a counterexample.

Turn intent into a contract

Precondition: caller's obligation

$x \leq limit$

Postcondition: callee's promise

$r \leq limit$

$$r = \begin{cases} x + 1 & x < limit \\ x & x = limit \end{cases}$$

The contract says both:

- what inputs are allowed;
- exactly how the result relates to the input.

A bound alone is too weak: returning zero would satisfy it.

Hoare triples are statements about executions

Let s and s' denote program states, and let $C(s, s')$ mean that program C can terminate in s' when started in s.

$$\{P\}\ C\ \{Q\} \quad \Longleftrightarrow \quad \forall s, s',\ P(s) \wedge C(s, s') \Longrightarrow Q(s').$$

- P: a predicate on the initial state (the precondition).
- Q: a predicate on the final state (the postcondition).
- Initial values can be retained in the state to express input–output relations.

Partial versus total correctness

This statement constrains terminating executions.
Total correctness additionally requires termination on permitted inputs.

From assignments to proof obligations

To establish Q after an assignment $x := e$, establish $Q[e/x]$ before it:

$$\{Q[e/x]\}\ x := e\ \{Q\}.$$

For example, with $Q(x) \equiv x \leq L$ and $e = x + 1$,

$$Q[e/x] \equiv x + 1 \leq L.$$

A branch supplies a hypothesis

Over $\mathbb{Z}$, the guard $x < L$ implies $x + 1 \leq L$.
The guard $x \leq L$ does not.

The statement of the theorem matters

- **Too weak:** $0 \leq r \leq L$ is also satisfied by $r = 0$.
- **Vacuous:** an inconsistent P makes $P \Rightarrow Q$ valid for any Q.
- **Wrong structure:** arithmetic in $\mathbb{Z}$ differs from arithmetic modulo 2^{32}.

Two separate questions

Does the implementation satisfy the formal specification?
Does the formal specification express the intended mathematics?

Z3: satisfiability relative to a theory

SMT: satisfiability modulo theories

Given a formula φ and background theory T, is there a model $\mathcal{M} \models T$ and an assignment α such that $\mathcal{M}, \alpha \models \varphi$?

- Boolean combinations of constraints in supported theories.
- Examples: integer arithmetic, fixed-width bit-vectors, arrays.
- The theory determines the meaning of symbols such as $+$ and $\leq$.

For our example: linear constraints over the integers.

Source: Microsoft's Z3 Guide

Prove a property by searching for its violation

$$\neg\exists s, s' : \; P(s) \wedge C(s, s') \wedge \neg Q(s').$$

Ask whether the existential counterexample formula is satisfiable.



- `sat`: a violation exists in this encoding.
- `unsat`: no such violation exists in this encoding.
- `unknown`: the solver could not decide; no proof obtained.

Source: Z3 Guide: Basic Commands

Ask Z3 for the boundary bug

```
(set-logic QF_LIA)
(declare-const x Int)
(declare-const limit Int)
(assert (and (<= 0 x) (<= x limit)
             (<= limit 4294967295)))
(define-fun r () Int (ite (<= x limit) (+ x 1) x))
(assert (> r limit)) ; violate the bound
(check-sat)
(get-model)
```

`sat`: for example, $x = 0$, $limit = 0$ gives $r = 1$.

Change the branch condition; check again

```
(define-fun r () Int (ite (< x limit) (+ x 1) x))
(assert (> r limit))
(check-sat)
; unsat
```

- In the increment branch: $x < \mathit{limit}$ implies $x + 1 \leq \mathit{limit}$.
- Otherwise: $x \leq \mathit{limit}$ implies $x = \mathit{limit}$.
- The input bound also gives $r \leq 2^{32} - 1$.

This query proves the bound. A complete contract also fixes the exact result.

Automation depends on the logical fragment

- Our query uses quantifier-free linear integer arithmetic.
- General quantified formulas require more than arithmetic simplification.
- Nonlinear arithmetic can also make proof search difficult.
- Auxiliary lemmas expose intermediate facts the search needs.

Failure to prove is not a disproof

A countermodel, an inconclusive answer, and a timeout are different outcomes.

Source: Z3 Guide: Quantifiers

Enough Rust to read the example

```
fn advance(x: u32, limit: u32) -> u32 {
    if x < limit { x + 1 } else { x }
}
```

- `fn`: define a function; `-> u32`: its output type.
- `u32`: an integer in $\{0, \ldots, 2^{32} - 1\}$.
- `if ... else ...`: choose an expression; the final expression is returned.

Finite arithmetic introduces an obligation

To execute `x + 1` without overflow, prove $x < 2^{32} - 1$.

Verus connects Rust code to proof obligations



- Write specifications and proofs alongside executable code.
- Verify supported Rust code with annotations and library specifications.
- Erase specification/proof code when compiling the executable.

Source: Verus project

Source: Verus Guide: code modes

Three modes, three jobs

Mode	Purpose	Example
`exec`	Actual computation	Update a counter
`spec`	Mathematical definitions	Describe the desired result
`proof`	Establish logical facts	Supply a lemma or assertion

- Think: computation, definition, and lemma.
- Specifications can use mathematical `int` and `nat`.
- Ghost code is reasoning-only code, erased before execution.

Source: Verus Guide: code modes

The counter contract in Verus

```
fn advance(x: u32, limit: u32) -> (r: u32)
    requires x <= limit,
    ensures
        r <= limit,
        r == (if x < limit { x + 1 } else { x }),
{
    if x < limit { x + 1 } else { x }
}
```

- `requires`: what callers must establish.
- `ensures`: what callers may rely on afterward.
- Verus also checks the executable addition for overflow.

Source: Verus Guide: contracts

What must be established?

1. **Safe addition:** $x < \mathit{limit} \leq$ `u32::MAX` gives room to increment.
2. **Bounded result:** each branch returns at most the limit.
3. **Exact behavior:** increment below the limit, stay put at the limit.

Try the original bug

Replace `x < limit` with `x <= limit` in the body.
The same contract should reject the implementation.

Using a verified function is applying a lemma

Suppose we have established

$$\forall x, L \in \{0, \ldots, 2^{32} - 1\}, \quad x \leq L \implies \text{advance}(x, L) = \min(x + 1, L).$$

- Instantiate at $(x, L) = (4, 5)$ to deduce the result is 5.
- Instantiate at $(x, L) = (5, 5)$ to deduce the result is again 5.
- At $(x, L) = (6, 5)$, the hypothesis fails; the theorem gives no guarantee.

Modular proof

Subsequent arguments use the function's contract as a lemma.
They need not repeat the proof about its implementation.

A loop invariant is an induction hypothesis

For `while` B `do` C, choose a predicate I on states.

$$\underbrace{P \Rightarrow I}_{\text{base case}}, \qquad \underbrace{\{I \wedge B\}\ C\ \{I\}}_{\text{inductive step}}, \qquad \underbrace{I \wedge \neg B \Rightarrow Q}_{\text{conclusion at exit}}.$$

For repeated bounded successor, take $I(x) \equiv 0 \leq x \leq L$.

- Initially: $0 \leq x_0 \leq L$ is assumed.
- If $x < L$, then $x' = x + 1 \leq L$; the invariant is preserved.
- At exit: $x \leq L$ and $\neg(x < L)$ imply $x = L$.

Termination uses well-foundedness

$V(x) = L - x \in \mathbb{N}$ decreases strictly on every iteration.

Loops need an invariant

```
fn fill_to_limit(start: u32, limit: u32) -> (r: u32)
    requires start <= limit,
    ensures r == limit,
{
    let mut x = start;
    while x < limit
        invariant x <= limit,
        decreases limit - x,
    {
        x = advance(x, limit);
    }
    x
}
```

Invariant: we stay within the limit. Variant: the remaining distance shrinks.

Source: Verus Guide: Loops and invariants

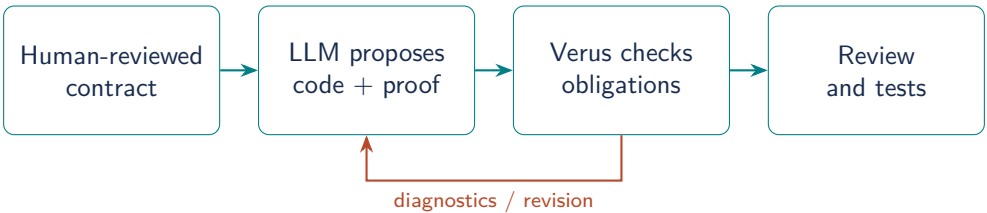
What is being trusted?

- The specification captures the intended theorem.
- The translation from code to logical obligations is sound.
- The solver's result and any assumed lemmas are correct.
- Compilation and execution preserve the verified meaning.

Distinguish proof search from proof checking

Here we rely on the Verus/Z3 verification pipeline.
We are not presenting an independently checked proof certificate.

An LLM proposes a construction; verification checks it



Plausibility is not a proof: check the generated construction against a fixed claim.

A concrete revision loop

1. Freeze the counter contract and allowed input domain.
2. Ask the LLM to implement the function and discharge obligations.
3. Reject the <= candidate when verification fails.
4. Revise the body to use <; check the same contract again.
5. Inspect the changed argument and test the surrounding program.

Helpful prompt constraint

"Keep the contract unchanged. Do not introduce assumptions or bypass verification. Explain any remaining obligations."

Do not let a proof attempt change the theorem

- **Weaken the conclusion:** replace $r = \min(x + 1, L)$ by $r \leq L$.
- **Strengthen the hypotheses:** exclude the failing boundary case.
- **Assume the claim:** introduce an unproved premise that implies it.

The guardrail must preserve the statement

Fix the contract, audit new assumptions, and require all intended obligations to pass.

The guardrail has a defined reach

Can establish, when specified

- Results satisfy a contract.
- Arithmetic stays in range.
- A modeled invariant is preserved.

Still needs separate evidence

- The contract matches user intent.
- External systems match the model.
- The algorithm is efficient enough to use.

Stronger acceptance criteria make generated code easier to evaluate.

Start with one property worth keeping true

1. Write the contract before polishing the implementation.
2. Use Z3's counterexample-search model to understand proof obligations.
3. Let Verus connect those obligations to Rust code.
4. Make verification a required check for LLM-generated changes.

Which theorem about an algorithm would you want its implementation to carry?

References and demo files

- Verus Tutorial and Reference

  `https://verus-lang.github.io/verus/guide/`

- Verus source and installation instructions

  `https://github.com/verus-lang/verus`

- Microsoft's Z3 Guide

  `https://microsoft.github.io/z3guide/`

Local examples: `examples/counter.rs`, `examples/counter-bug.smt2`, `examples/counter-fixed.smt2`.